

saleshealth — Análisis de Valor de Vida del Cliente (CLTV)

Documento Técnico

Asignatura: Gestión de Datos · UAX

Fecha: 11 de mayo de 2026

Repositorio: [GD_proyecto_final/](#)

1. Introducción y Contexto

Resumen ejecutivo

El proyecto construye un sistema analítico end-to-end sobre datos transaccionales de una cadena de venta de productos de salud en Madrid. A partir de 17 tablas operacionales (3 esquemas PostgreSQL, 71.934 filas en origen), se implementa un pipeline ETL que genera un modelo dimensional Kimball con 2 tablas de hechos y 7 dimensiones, seguido de una mart `customer_360` con 26 indicadores por cliente. Sobre esa base se realizan análisis de distribución CLTV, clustering no supervisado y un dashboard interactivo de 5 páginas.

Números clave: 5.750 clientes activos · 42.313 líneas de venta · 9.341.505,20 € ingresos netos · 3.670.369,64 € de valor de vida total (CLTV) · K=4 segmentos con silhouette=0,696.

Hallazgos principales:

- La distribución del CLTV es **bimodal**, no Pareto continua: dos poblaciones casi disjuntas separadas por un valle vacío entre 500 € y 2.000 €.
- Concentración extrema:** el 13 % de clientes (recurrentes) genera el 91,5 % del CLTV positivo (Gini=0,830).
- Hallazgo contraintuitivo:** `margin_rate` correlaciona negativamente con CLTV ($\rho=-0,67$): mayor margen porcentual por unidad no implica mayor valor total.
- Sub-cluster anómalo** de 7 clientes con CLTV negativo detectado automáticamente por K-Means, replicando el hallazgo manual previo (validación de robustez).

Fuente de datos

Sistema operacional saleshealth organizado en **4 subsistemas**:

Total: **17 tablas**, **71.934 filas** en origen, **3 columnas con nulos** (`brand.website`, `city_zone.zone_orientation`, `sale_item.offer_id`).

Arquitectura del sistema — 3 capas

Stack tecnológico

2. Modelo Dimensional

Esquema estrella

El DWH implementa un **esquema estrella Kimball** con 2 tablas de hechos y 7 dimensiones compartidas:

```
dim_fecha —————
|
|
dim_zona — dim_tienda — fact_ventas — dim_cliente — dim_zona
|
|
dim_producto dim_oferta
|
dim_motivo — fact_devoluciones — dim_cliente
|
dim_fecha (role-playing: return_date / venta_date)
```

`dim_zona` se referencia desde `dim_tienda` (zona de la tienda) y `dim_cliente` (zona predominante del cliente, calculada como moda del distrito de compras).

Tabla de hechos principal — `fact_ventas`

Ground truth: 42.313 filas, 9.617.816,01 € brutos.

Decisiones de diseño relevantes

- **Surrogate keys:** `dim_fecha` usa INT YYYYMMDD (smart key legible en fact sin JOIN); resto de dimensiones usan BIGINT IDENTITY.
- **Sentinel rows:** `dim_oferta` (`oferta_id=0`) y `dim_motivo` (`motivo_id=0`) permiten FK NOT NULL en todas las tablas de hechos. El 99,98 % de las líneas de venta no tienen oferta; el sentinel elimina NULLs sin coste semántico.
- **SCD Tipo 1:** todas las dimensiones. Cambios en atributos sobrescriben el valor anterior (no se guarda histórico de cambios).
- **Idempotencia ETL:** el orquestador ejecuta TRUNCATE antes de cada INSERT por lote; el script puede lanzarse N veces sin duplicar datos.
- `fact_devoluciones` **independiente de** `fact_ventas` (Opción A Kimball): no hay FK entre hechos; la trazabilidad se mantiene vía `sale_item_id` como degenerate dimension.

3. ETL y Calidad de Datos

Pipeline ETL — orden topológico

Tiempo total de ejecución: 12,2 segundos (PostgreSQL 18, hardware estándar).

Decisiones de limpieza — D01 a D07

Verificación de cuadros financieros

4. CLTV: Definición y Cálculo

Definición adoptada (D-CLTV-01)

CLTV histórico = valor real generado, no proyección futura. Permite segmentación y sirve de variable objetivo para modelos predictivos en fases posteriores.

Fórmula y componentes

```
CLTV = margen_neto = SUM(margen_bruto) - SUM(importe_devuelto) + SUM(costo_devuelto)
```

La corrección por `costo_devuelto` es crítica (D-CLTV-04): sin ella, una devolución completa produciría margen neto negativo en lugar de cero.

Verificación unitaria:

- Venta: precio=100 €, costo=60 €, margen_bruto=40 €
- Devolución completa: importe_devuelto=100 €, costo_devuelto=60 €
- Fórmula corregida: $40 - 100 + 60 = 0 \text{ €}$ | Fórmula incorrecta: $40 - 60 = -20 \text{ €}$

Identidad algebraica de los componentes (D-CLTV-13)

Los cuatro componentes descriptivos del CLTV satisfacen:

```
AOV * margin_rate * freq_mensual * meses_activo = margen_bruto_total
```

Demostración algebraica:

Verificación empírica (D-CLTV-14): $\max|\text{margen_bruto} - \text{ROUND}(\text{producto}, 2)| = 1,41 \text{ €}$ sobre 3.670.369,64 € → error relativo $3,8 \times 10^{-7}$ (< 1 ppm).

Cota teórica del error de representación NUMERIC(8,4): $\epsilon_{\text{freq}} \leq 0,00005 \rightarrow \max_error \approx \text{AOV_max} \times \text{margin_rate} \times \epsilon_{\text{freq}} \times \text{meses_max} \approx 1.000 \times 0,40 \times 0,00005 \times 72 \approx 1,44 \text{ €}$. El resultado empírico (1,41 €) cae dentro de la cota teórica; el umbral de alerta se fija en 2,00 € (margen del 39 %).

Resultados del CLTV

5. Distribución y Concentración

Distribución del CLTV

La distribución del CLTV **no sigue una ley de potencias (Pareto) continua**. Presenta **bimodalidad estructural**: dos poblaciones casi disjuntas separadas por un valle vacío entre 500 € y 2.000 €.

Estadísticos de forma:

- **Skewness (Fisher):** 2,57 — asimetría positiva moderada para retail (la bimodalidad, no la cola larga, explica el valor; la cola sería >4 en distribución Pareto pura)

- **Exceso de curtosis:** 5,37 — colas más pesadas de lo normal

El boxplot Tukey clasifica el 13 % superior como *outliers*, pero son una **población estructural** (recurrentes premium), no anomalías estadísticas. Decisión metodológica: no eliminar, segmentar.

Percentiles clave:

El salto entre p75 (120 €) y p90 (3.555 €) confirma el valle vacío y la bimodalidad.

Concentración — Curva de Lorenz

Gini = 0,830 sobre CLTV positivo (n=5.444). Alta concentración estructural.

Correlaciones Spearman con CLTV

Se utiliza Spearman (no Pearson) por skewness=2,57, exceso de curtosis=5,37 y bimodalidad visual, que invalidan el supuesto de normalidad marginal de Pearson.

La correlación negativa de `margin_rate` tiene explicación de negocio: los clientes recurrentes compran productos de mayor volumen y precio pero menor margen porcentual; los transaccionales compran artículos sueltos con mayor margen relativo.

6. PCA y Clustering

Selección de variables (features)

Variables descartadas:

PCA — análisis de varianza

StandardScaler aplicado a las 4 features post-transformación. PCA sobre `x_scaled` para análisis de varianza:

Decisión (D-CLU-05/06): K-Means y Silhouette operan sobre `x_scaled` (espacio original de 4 variables). `x_pca[PC1, PC2]` se reserva exclusivamente para visualización. Con 4 variables sin ruido, reducir a PCA no aporta separación adicional.

Validación ARI: clustering sobre `x_scaled` vs clustering sobre `x_pca[PC1, PC2]` → ARI=0,358. Interpretación: proyectar a 2 dimensiones descarta el 35,7 % de varianza relevante, alterando significativamente las asignaciones. Justifica el uso del espacio original.

Selección de K — método Silhouette

K=4 maximiza el Silhouette score (rango 0–1; >0,5 indica segmentos bien diferenciados). La bimodalidad observada en Fase 5 (87 % transaccionales + 13 % recurrentes) predecía K≥2; K=4 añade la detección de devolvedores y el sub-cluster anómalo.

7. Los 4 Segmentos

Tabla resumen de segmentos

Medianas por cluster excepto CLTV medio (media) y CLTV total (suma). Datos exactos del notebook `06_clustering.ipynb`.

Descripción y recomendación por segmento

Recurrentes premium (13 % de clientes, 91,5 % del CLTV positivo)

Compran con alta frecuencia (18 meses activos de media), ticket elevado (580 €) y tasa de devolución mínima (2 %). Son el núcleo generador de valor. **Acción:** retener a toda costa; programa de fidelización, atención prioritaria, ofertas exclusivas anticipadas.

Compradores únicos (79,5 % de clientes, 8,3 % del CLTV)

Un solo pedido o frecuencia muy baja. CLTV modesto pero masa crítica enorme (4.573 clientes). **Acción:** campaña de segunda compra personalizada en los 30–60 días post-venta; el coste de activación es bajo comparado con el potencial de conversión al segmento premium.

Devolvedores (7,3 % de clientes, 0,1 % del CLTV)

Frecuencia normal pero $\text{return_rate}=1,00$ (prácticamente todo lo comprado se devuelve). Generan coste operativo elevado con retorno mínimo (12 € de CLTV medio). **Acción:** revisión individualizada; restricciones en política de devoluciones; análisis de si el problema es el producto o el perfil del cliente.

Productos a pérdida (0,1 % de clientes, CLTV negativo)

7 clientes con margin_bruto negativo y sin devoluciones registradas: el coste del producto supera el precio de venta. margin_rate mediana = $-2,00$ (pérdida del 200 %). **Acción:** investigación caso a caso; posible error de precio en catálogo o fraude en devoluciones no registradas.

Hallazgo de validación: El sub-cluster "Productos a pérdida" fue detectado automáticamente por K-Means. El análisis manual previo (Fase 5, Sección 0) había identificado estos 7 clientes como anomalía. La coincidencia exacta valida la robustez del modelo de clustering.

8. Dashboard y Visualización

Arquitectura del dashboard — 5 páginas Streamlit

Decisiones de diseño UX

- **Regla de negocio vs técnico:** tecnicismos (silhouette, ARI, PCA, surrogate key) exclusivamente en expanders colapsables. Detalles técnicos. La pantalla principal usa únicamente lenguaje de negocio.
- **Colores por segmento:** consistentes en todas las páginas (#2ECC71 premium, #3498DB únicos, #E67E22 devolvedores, #95A5A6 pérdida).
- **Caché:** `@st.cache_data(ttl=600)` para queries frecuentes, `ttl=3600` para metadatos (volúmenes, rango de fechas).
- **Visualización:** Plotly Express y Graph Objects para toda representación gráfica. `go.Scatterpolar` para radar charts 2x2. `go.Histogram` con escala log pre-computada en NumPy para evitar bugs de Plotly con NUMERIC PostgreSQL en eje logarítmico.

Stack de visualización y arquitectura

```
app.py (Página 1) — utils.py (engine, cache, formatters)
pages/
2_Modelo_de_Datos.py ← get_rango_fechas(), get_ventas_por_anio()
3_Analisis_CLTV.py ← load_data() → customer_360
4_Segmentacion.py ← load_data(sql) + radar charts
5_Customer_360.py ← load_data() + session_state + percentiles
```

Arranque: `streamlit run src/dashboard/app.py` desde la raíz del proyecto.

9. Conclusiones y Limitaciones

Conclusiones

1. Bimodalidad estructural, no distribución Pareto

La distribución del CLTV presenta dos poblaciones casi disjuntas separadas por un valle vacío (500 €–2.000 €). $\text{Skewness}=2,57$ y $\text{Gini}=0,830$. El análisis de clustering confirma la estructura bimodal: $K=4$ identifica dos clusters claramente diferenciados antes de llegar a la cola alta.

2. Identidad algebraica verificada con análisis de error de representación

$\text{AOV} \times \text{margin_rate} \times \text{freq_mensual} \times \text{meses_activo} = \text{margin_bruto_total}$ con error máximo empírico de 1,41 € sobre 3,67 M€ ($3,8 \times 10^{-7}$). La cota teórica (NUMERIC(8,4), 1,44 €) explica el error completamente, confirmando que no hay bug de implementación.

3. Concentración extrema: top 13 % genera 91,5 % del valor

El coeficiente de Gini (0,830) es elevado incluso para retail. El segmento Recurrentes premium (750 clientes) concentra 3.359.998 € de los 3.670.370 € totales. La dependencia de un grupo tan pequeño constituye el principal riesgo de negocio identificado.

4. margin_rate negativamente correlado con CLTV ($\rho=-0,67$)

Los clientes de mayor valor compran productos de mayor volumen/precio pero menor margen porcentual; los transaccionales compran productos sueltos con mayor margen unitario. Implicación: maximizar margen porcentual en la oferta puede ser contraproducente para capturar clientes de alto valor.

5. Sub-cluster anómalo detectado automáticamente — validación de robustez

Los 7 clientes con CLTV negativo (detectados manualmente en Fase 5) fueron recuperados exactamente por K-Means como cuarto cluster. La concordancia exacta valida que el modelo no es sensible a la inicialización aleatoria ni a los hiperparámetros elegidos.

6. $\text{ARI}=0,358$ cuantifica pérdida de información al reducir dimensionalidad

La diferencia entre asignaciones en espacio original (X_{scaled}) y en espacio reducido (PC1+PC2, 64,3 % de varianza) es significativa: $\text{ARI}=0,358$ implica que el 35,7 % de varianza descartada contiene información discriminativa real. Justifica el uso del espacio original para clustering, reservando PCA para visualización únicamente.

Limitaciones

Trabajo futuro

1. **Modelo predictivo de churn** para recurrentes premium: regresión logística o supervivencia (Cox) con las features de `customer_360`.
2. **Segmentación dinámica con actualización mensual** vía pipeline programado: `run_etl.py` + re-clustering sobre ventana deslizante.

3. **Análisis de cesta de la compra** por segmento: reglas de asociación (Apriori) sobre fact_ventas para identificar patrones de co-compra entre productos de salud.

4. **Incorporación de variables geográficas** (zona_id, tipo_area) como features en el modelo de clustering para detectar heterogeneidad espacial.

10. Detalles Técnicos del ETL

Scripts principales y transformaciones

El ETL se ejecuta vía python src/etl/run_etl.py, que orquesta 9 pasos secuenciales usando SQLAlchemy para conexiones tipadas.

Ejemplo de transformación en dim_cliente (paso 7):

```
INSERT INTO dwh.dim_cliente (
cliente_id, nombre, email, fecha_alta, zona_predominante_id,
aov, margin_rate, meses_activo, n_pedidos, return_rate, cltv
)
SELECT
c.customer_id,
c.name,
c.email,
MIN(s.sale_date) AS fecha_alta,
MODE() WITHIN GROUP (ORDER BY z.zone_id) AS zona_predominante_id,
AVG(s.total / si.quantity) AS aov,
SUM(si.subtotal - si.quantity * cp.unit_cost) / SUM(si.subtotal) AS margin_rate,
EXTRACT(EPOCH FROM MAX(s.sale_date) - MIN(s.sale_date)) / 2592000 AS meses_activo,
COUNT(DISTINCT s.sale_id) AS n_pedidos,
COALESCE(SUM(CASE WHEN r.return_id IS NOT NULL THEN 1 ELSE 0 END) / COUNT(DISTINCT s.sale_id), 0) AS return_rate,
SUM(si.subtotal - si.quantity * cp.unit_cost) - COALESCE(SUM(r.quantity * cp.unit_cost), 0) AS cltv
FROM public.customer c
JOIN public.sale s ON c.customer_id = s.customer_id
JOIN public.sale_item si ON s.sale_id = si.sale_id
JOIN central_product cp ON si.product_id = cp.product_id
LEFT JOIN public.return_item r ON si.sale_item_id = r.sale_item_id
LEFT JOIN public.store st ON s.store_id = st.store_id
LEFT JOIN public.city_zone z ON st.postal_code = z.postal_code
GROUP BY c.customer_id, c.name, c.email;
```

Optimizaciones implementadas:

- **Batching:** Procesamiento en lotes de 1.000 filas para evitar memory overflow.
- **Índices temporales:** Creación de índices en tablas staging durante carga.
- **Validaciones:** CHECK constraints en tablas de hechos para integridad (e.g., quantity > 0).

Manejo de errores y logging

- **Try-except blocks:** Captura de excepciones SQL con rollback automático.
- **Logging:** Uso de logging module con niveles INFO/ERROR; salida a etl.log.
- **Idempotencia:** Scripts verifican existencia previa antes de INSERT para evitar duplicados.

11. Esquemas de Base de Datos y Consultas

Esquema DWH (PostgreSQL)

Tabla fact_ventas:

```
CREATE TABLE dwh.fact_ventas (
venta_id BIGINT PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
date_id INT NOT NULL REFERENCES dwh.dim_fecha(date_id),
cliente_id BIGINT NOT NULL REFERENCES dwh.dim_cliente(cliente_id),
producto_id BIGINT NOT NULL REFERENCES dwh.dim_producto(producto_id),
tienda_id BIGINT NOT NULL REFERENCES dwh.dim_tienda(tienda_id),
oferta_id BIGINT NOT NULL REFERENCES dwh.dim_oferta(oferta_id),
sale_id INT NOT NULL,
sale_item_id INT NOT NULL,
quantity INT NOT NULL CHECK (quantity > 0),
unit_price NUMERIC(10,2) NOT NULL,
unit_cost NUMERIC(10,2) NOT NULL,
subtotal NUMERIC(12,2) NOT NULL CHECK (subtotal >= 0),
margen_bruto NUMERIC(12,2) NOT NULL
);
```

Consulta ejemplo: CLTV por zona

```
SELECT
z.nombre_zona,
COUNT(DISTINCT c.cliente_id) AS n_clientes,
SUM(c.cltv) AS cltv_total,
AVG(c.cltv) AS cltv_medio
FROM dwh.dim_cliente c
JOIN dwh.dim_zona z ON c.zona_predominante_id = z.zona_id
GROUP BY z.nombre_zona
ORDER BY cltv_total DESC;
```

Rendimiento y optimización

- **Particionamiento:** fact_ventas particionada por date_id (mensual) para queries temporales.
- **Materialized views:** customer_360 como MV para acelerar dashboard.
- **Vacuum y analyze:** Automático post-ETL para mantener estadísticas actualizadas.